Reproducing USAD on the Server Machine Dataset and Testing It on New Multivariate Data

Abstract

I reproduce USAD (Audibert et al., 2020), an unsupervised anomaly detector for multivariate time series built from two adversarially trained autoencoders, and test how well it generalises. Running the unmodified official code on all 28 machines of the Server Machine Dataset (SMD), I obtain F1* = 0.763 with point adjustment against 0.946 in the paper, and 0.528 without it. I trace most of the gap to untuned hyperparameters and a protocol that the paper leaves partly unspecified. On two new datasets, the Skoltech Anomaly Benchmark (SKAB) and a dataset I constructed by injecting 80 labelled anomalies into real household power readings, USAD's point-adjusted scores look strong on SKAB (F1* 0.973) but its ranking quality is near chance on most experiments (mean AUC 0.671), and on the constructed data it catches dropouts but almost none of the spikes or correlation breaks. Finally, an LLM used as a judge of USAD's alerts agrees closely with the injected ground truth (Cohen's κ = 0.88) and, used as a filter, raises alert precision from 0.25 to 0.92.

1 Description of the Paper and Justification

1.1 The problem and the method

Servers, water plants and spacecraft are monitored by many sensors that report together over time, forming a *multivariate time series*. The task is to flag time points where the system behaves abnormally, *without* labelled faults for training, because real faults are rare and costly to label.

USAD[1] (Audibert et al., 2020) learns only from normal data. It uses one shared encoder E and two decoders D_1, D_2, giving two autoencoders $AE_1 = D_1(E(\cdot))$ and $AE_2 = D_2(E(\cdot))$, applied to sliding windows W of the series. Training has two phases weighted by the epoch n. Both autoencoders first learn to reconstruct W. Then AE_2 learns to tell real windows from AE_1's reconstructions, while AE_1 learns to fool it. Writing $r_1 = \|W - AE_1(W)\|^2$, $r_2 = \|W - AE_2(W)\|^2$ and $r_{12} = \|W - AE_2(AE_1(W))\|^2$:

$$\mathcal{L}_{AE_1} = \tfrac{1}{n}\, r_1 + \left(1 - \tfrac{1}{n}\right) r_{12}$$
$$\mathcal{L}_{AE_2} = \tfrac{1}{n}\, r_2 - \left(1 - \tfrac{1}{n}\right) r_{12}$$

At test time, the anomaly score is $\alpha\, r_1 + \beta\, r_{12}$. The authors argue that this adversarial step amplifies the reconstruction error of abnormal windows while avoiding the instability of a full GAN such as MAD-GAN (Li et al., 2019).

1.2 Evaluation framework

USAD is scored with point-wise Precision, Recall and F1 over anomaly labels. For SMD, SMAP and MSL, the paper follows the OmniAnomaly protocol (Su et al., 2019):

- The threshold is chosen by searching for the best F1.
- *Point adjustment* (PA) is applied (Xu et al., 2018): if any point of a true anomalous segment is flagged, the whole segment counts as detected.
- F1* is reported alongside F1, computed from Precision and Recall averaged over the entities (here, the 28 machines).

PA matters a great deal. For a six-point segment where only two points are flagged, recall is 0.33 without PA and 1.0 with it. Later work shows that PA can make even random scores look competitive (Kim et al., 2022), so I report every result both with and without PA.

On SMD the paper reports P = 0.931, R = 0.962 and F1 = 0.938, which gives F1* = 0.946. These are among the best results in its comparison, and USAD has since become a standard baseline.

[1] https://dl.acm.org/doi/10.1145/3394486.3403392

Dataset	Dims	Entities	Access
SWaT	51	1	on request (iTrust)
WADI	123	1	on request (iTrust)
SMD	38	28	open (MIT licence)
SMAP	25	55	open
MSL	55	27	open

Table 1: The five datasets used by Audibert et al. (2020): SWaT (Mathur and Tippenhauer, 2016), WADI (Ahmed et al., 2017), SMD (Su et al., 2019), SMAP and MSL (Hundman et al., 2018). I replicate on SMD.

1.3 Justification

- **Venue.** KDD is ranked A* in CORE.
- **Impact.** The paper had about 845 citations on Semantic Scholar when I wrote my proposal, and later detectors such as the Anomaly Transformer use it as a baseline (Xu et al., 2022).
- **Code.** The official implementation is public under BSD-3 and has third-party forks.[2]
- **Data.** SMD, unlike SWaT and WADI, is open with no access request, so the replication stays inside the paper's own benchmark.
- **Compute.** Training needs well under a minute per machine on a free GPU.

2 Description of the Original Datasets

The paper uses five benchmarks (Table 1). SWaT and WADI come from water-treatment and water-distribution testbeds, with attacks staged by the testbed operators. SMAP and MSL are NASA spacecraft telemetry. **SMD**[3] contains five weeks of one-minute resource metrics (CPU, memory, disk, network) from 28 servers in three groups of a large internet company. For each machine, SMD provides three plain-text files with one row per minute:

- `train/`: unlabelled, mostly normal data with 38 min-max-normalised comma-separated values per row.
- `test/`: data in the same format.
- `test_label/`: one 0/1 label per test row.

[2] https://github.com/manigalati/usad

[3] https://github.com/NetManAIOps/OmniAnomaly/tree/master/ServerMachineDataset

```
train/machine-1-1.txt (first 5 of 38 columns)
0.032258,0.039195,0.027871,0.024390,0.000000,...
0.043011,0.048729,0.033445,0.025552,0.000000,...
test_label/machine-1-1.txt
0
0
```

Figure 1: SMD format: normalised metrics, one row per minute; labels in a separate file.

Each half of a machine's data has 23,688–28,743 rows. On average 4.2% of test points are anomalous, ranging from 0.4% to 15.7% across machines. Figure 1 shows a snippet.

3 Replication of the Original Work

Environment. The official repository contains only `usad.py` (the model and training loop), `utils.py`, and a notebook built around SWaT, with no SMD pipeline. My earlier local Windows/Jupyter setup ran into PyTorch installation problems, so I ran the official code *unmodified* in Google Colab on a T4 GPU with Colab's preinstalled PyTorch. I wrote my own SMD loader around it. All scripts and notebooks are in my repository.[4] I also re-ran the constructed-dataset experiment on CPU with PyTorch 2.15 and got the same results to two decimals (Section 5).

Protocol. For each machine:

- Min-max scaling is fitted on the training file only.
- Sliding windows have length 12, and the latent size is 12×100, as in the official notebook.
- Training runs for 100 epochs with Adam on 80% of the training windows; the other 20% are used for validation.
- Scoring uses $\alpha = \beta = 0.5$, and the seed is fixed at 0.
- A test window is labelled anomalous if any point inside it is.
- The threshold maximising F1 is chosen from 300 quantiles of the score, between the 50th and 99.99th percentiles, separately with and without PA.
- F1* is computed from Precision and Recall averaged over machines, as in the paper.

[4] https://github.com/vishnuprasad033/usad-comp8240

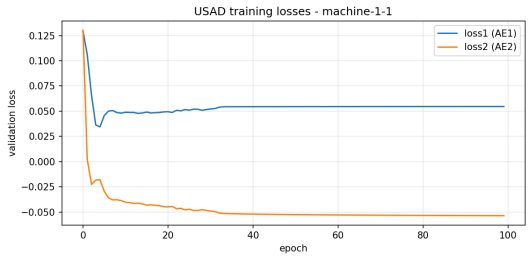


Figure 2: Validation losses of AE_1 and AE_2 on SMD machine-1-1. Both converge smoothly; $\mathcal{L}_{AE_2}$ goes negative by design.

		P	R	F1	F1*
Paper	PA	0.931	0.962	0.938	0.946
Mine	PA	0.792	0.736	0.723	0.763
	no PA	0.591	0.476	0.461	0.528

Table 2: USAD on all 28 SMD machines. P, R, F1 are means over machines; F1* is the F1 of mean P and R. Mean ROC AUC is 0.772.

Training took about 20 seconds per machine. Both losses converged smoothly with no GAN-style oscillation (Figure 2), which matches the paper's stability claim.

Results. Table 2 compares my results with the paper. Replication is *partial*: with PA, I reach F1* = 0.763 against 0.946 in the paper. Figure 3 shows that the mean hides a wide spread:

- 9 of 28 machines reach F1 ≥ 0.9 with PA, and two (2-8 and 3-11) reach 1.0.
- 7 machines score below 0.5; the lowest is machine 3-3 at 0.27.
- Without PA, every machine drops, often by half. Machine 1-1 falls from 0.92 to 0.40, and F1* falls to 0.528.

Figure 4 shows why for machine 1-1: the score rises inside most labelled segments but crosses the threshold for only part of each one, which PA then credits as a full detection.

Why the results are not closer. I see five plausible causes, in rough order of importance.

1. **Hyperparameters.** The paper tunes the window size and the α/β weights per dataset. I used the notebook defaults built for SWaT, and the paper itself reports a sensitivity analysis for these parameters.
2. **Protocol details the paper does not fully specify.** It does not say how window scores map back to points, which threshold grid it searches, or exactly how F1 and F1* are aggregated. My window-level labelling (any anomalous point marks the window) is one reasonable choice among several.
3. **Threshold search.** The paper's search over thresholds may be finer than my 300 quantiles, and best-F1 search is sensitive to it.
4. **One seed.** I trained each machine once, while the large per-machine variance suggests results move between runs.
5. **Software drift.** The code targets PyTorch 1.6, while I ran a current version.

I found no error in the official code. The authors' later work also questions whether deep models clearly beat simple baselines on these benchmarks (Audibert et al., 2022), which is consistent with how fragile the headline numbers are.

4 Construction of New Data

4.1 Existing dataset: SKAB

SKAB[5] (Katser and Kozitsin, 2020) is an industrial water-circulation testbed recorded once per second with eight sensors (two accelerometers, current, pressure, two temperatures, voltage and flow rate). It contains one anomaly-free run of 9,405 rows and 34 labelled experiments (16 `valve1`, 4 `valve2`, 14 `other`). Together the experiments hold 37,401 rows, 35% of them anomalous, where faults such as valve closures and pump disturbances were staged.

SKAB is close in spirit to SWaT but needs no access request, so it fits the paper's industrial setting. It was not used in the paper. Each file is `;`-separated with an `anomaly` label per row (Figure 5). My preprocessing:

- I drop the `changepoint` column.
- I fit the scaler on the anomaly-free run, train one USAD model on it, and score every experiment file separately.
- Test values are clipped to $[-5, 5]$ after scaling, because some experiments go far outside the normal range.

[5] https://github.com/waico/SKAB

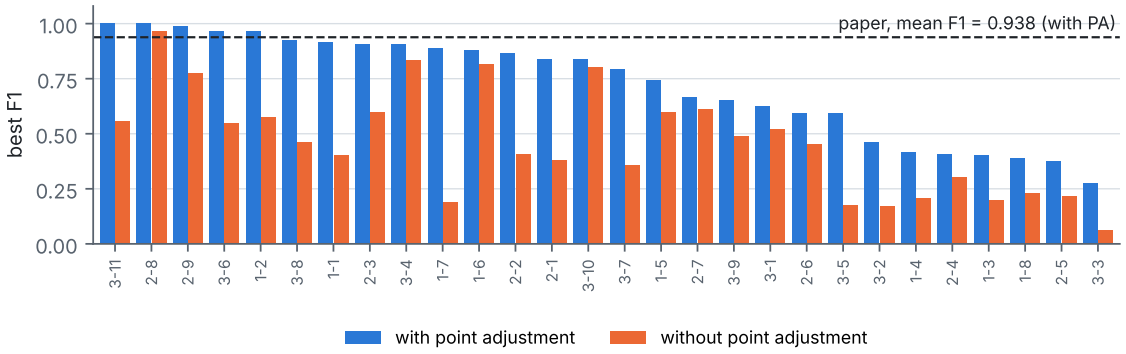


Figure 3: Best F1 per SMD machine, with and without point adjustment, sorted by the PA score. The dashed line is the paper's reported F1.

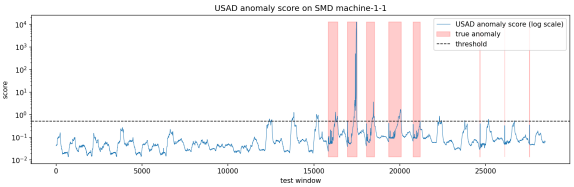


Figure 4: USAD score (log scale) on SMD machine-1-1 test data, with labelled anomalies (red) and the best-F1 PA threshold.

```
datetime;Accelerometer1RMS;...;Current;
  Pressure;Temperature;...;
  Volume Flow RateRMS;anomaly;changepoint
2020-03-09 10:14:33;0.0265878;...;1.3302;0.054711;
  79.3366;...;32.0;0.0;0.0
```

Figure 5: A row of SKAB `valve1/0.csv` (abridged).

Exathlon (Jacob et al., 2021), my second proposed candidate, was dropped for time. It needs a much heavier pipeline (93 high-dimensional Spark traces), and SKAB plus the constructed dataset already cover the proposal's two new-data requirements.

4.2 Constructed dataset: household power with injected anomalies

Real multivariate data with trustworthy anomaly labels is scarce, and labelling it by hand would need domain experts I do not have. I therefore built a dataset whose labels are correct *by construction*: I start from real but unlabelled readings and inject anomalies with known type, timing and severity. The pipeline is a fixed-seed script (seed 42) in the repository.

Collection. The UCI Individual Household Electric Power Consumption data[6] (Hebrail and Berard, 2012) has 2,075,259 one-minute readings from one house near Paris (Dec 2006 – Nov 2010). It has 7 channels: active and reactive power, voltage, current intensity, and three sub-meters for the kitchen, laundry and water heater/air-conditioner. 1.25% of rows are missing, and there are no anomaly labels. Its domain (smart-home IoT) differs from both SMD and SKAB.

Preprocessing. I resampled to 10-minute means, which gives 207,527 steps while keeping daily patterns. Gaps of up to one hour were filled by time interpolation. Longer outages were also interpolated but flagged, and kept out of injection. The data was then split by date into a clean training period (2007-01-01 to 2009-06-30, 131,328 steps) and a test period (2009-07-01 onwards, 73,999 steps).

Annotation by injection. Into the test period I injected 20 segments of each of four types, each type at low or high severity. Segments never overlap, are at least a day apart and avoid imputed outages.

- **Spike** (1–3 steps): a channel jumps by 3σ (low) or 6σ (high).
- **Drift** (6–24 hours): a linear offset that ramps up to 2σ or 4σ.
- **Dropout** (2–12 hours): one channel (low) or all channels (high) read 0, as in a meter failure.
- **Decoupling** (4–12 hours): current is replaced, 50% (low) or 100% (high), by its value from a different week. This breaks its physical link to active power while each channel still looks plausible on its own, much like the spoofed sensor in SWaT.

These 80 segments cover 4.59% of test steps, and every one is recorded in an injection log with type,

[6] https://doi.org/10.24432/C58K54

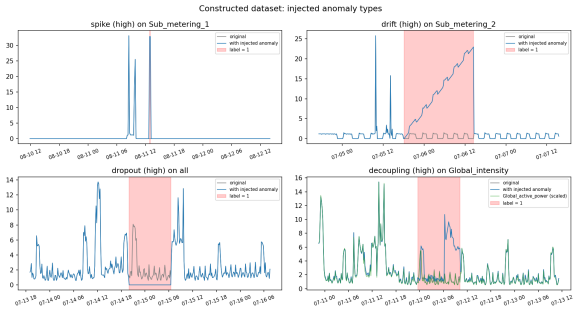


Figure 6: One high-severity example of each injected type: original (grey), after injection (blue), label (red band).

Data		P	R	F1 / F1*	AUC
SMD	PA	0.792	0.736	0.763*	0.772
	no PA	0.591	0.476	0.528*	
SKAB	PA	0.947	1.000	0.973*	0.671
	no PA	0.602	0.929	0.730*	
UCI	PA	0.538	0.433	0.480	0.639
	no PA	0.497	0.188	0.273	

Table 3: USAD with identical settings on the original and new data. * = F1*; UCI is a single series, so plain F1 is given.

channel, severity, start and end (Figure 6; snippet in Appendix A).

Quality and agreement. Labels come from the generator, so no human annotation was needed, and agreement with ground truth is exact by definition. I verified the dataset in two ways:

- Re-running the script outside Colab reproduced the injection log byte for byte.
- The LLM-as-judge study (Section 5.1) is an independent check that the injected anomalies are recognisable in the data. Its agreement with the injection log is $\kappa = 0.88$.

Compared with SMD, the constructed data is smaller and has synthetic anomalies, but it offers what SMD's expert labels cannot: per-type and per-severity ground truth.

5 Results on New Data

Both new datasets use exactly the replication's settings and evaluation code, so the numbers are directly comparable (Table 3). One difference: SKAB is scored per file and averaged, as SMD is per machine, while the UCI data is a single series.

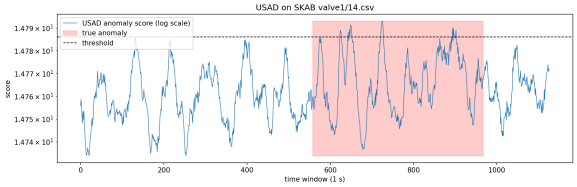


Figure 7: USAD score on a typical SKAB valve experiment: scores are high throughout, so the threshold cannot separate the labelled anomaly.

SKAB. With PA, SKAB looks almost perfect: F1* is 0.973, and 20 of 34 files score F1 = 1.0. This is mostly an artefact. SKAB's anomalies are long, about a third of each file, so flagging a single point in each segment earns full recall. AUC, which does not depend on a threshold, tells a different story:

- On the `other` experiments, mean AUC is 0.865 and USAD ranks anomalies well.
- On `valve1` and `valve2`, mean AUC is 0.528 and 0.561, close to chance (Figure 7).

The valve experiments seem to run in a different operating regime from the single anomaly-free run, so most of each file looks unfamiliar to the model, and normal and anomalous windows both score high. This is a distribution-shift failure that the paper's evaluation cannot reveal: on SMD, train and test come from the same machine and regime.

Constructed UCI dataset. This is the hardest of the three datasets, with F1 of 0.480 with PA and 0.273 without, and AUC of 0.639. The per-type breakdown (Figure 8a, at the best no-PA threshold) explains why; overall, USAD detects 24 of the 80 segments.

- **Dropouts:** every high-severity (all-channel) dropout is caught, but none of the single-channel ones.
- **Drifts:** 55% of high-severity and 22% of low-severity drifts are caught.
- **Spikes and decouplings:** almost none are caught (5% each).

The 10-minute averaging and the household's naturally large appliance spikes plausibly hide injected spikes. The decoupling result is the most informative. It is exactly the cross-channel inconsistency that a multivariate detector is meant to catch and that motivated the method in my proposal, yet USAD misses it: its reconstruction error is

dominated by each channel's own magnitude, and swapped-in current values stay within a normal range.

Compared with the replication, USAD generalises reasonably to data like its training distribution (the SKAB `other` files, AUC 0.865) but struggles with regime shift (SKAB valves) and with subtle multivariate anomalies (UCI). PA hides both problems.

5.1 LLM-as-judge

Design. I used an LLM to review USAD's alerts. This tests whether an LLM's judgement can stand in for, or add to, human review of detected segments, which is the costly step in real monitoring.

- At the best no-PA threshold, runs of flagged UCI windows (merged when fewer than 6 windows apart) give **110 alerts**. Matched against the hidden injection log, 27 are true anomalies and 83 are false alarms.
- For each alert, a script writes a prompt with the raw values of all seven channels during the alert and two hours either side (downsampled to 20 points), the median value for that hour of day in the training period, and the current/power ratio. It also gives a short, label-free description of the channels and their physics (Appendix B).
- Prompts carry no dates, labels or injection details, and items are shuffled with random IDs.
- The judge answers in JSON: `anomaly`/`normal`, a type, a 1–5 confidence and a one-sentence rationale.
- Verdicts were produced by Claude Opus 5.5 and scored against the key only after all 110 were written.

Results. The judge agrees closely with the ground truth: accuracy is 0.955, precision 0.923, recall 0.889, and Cohen's $\kappa = 0.88$ (Cohen, 1960). When the judge and the log both call an alert an anomaly, the judge names the right type in 23 of 24 cases. Its confidence is informative: all 21 confidence-5 verdicts were correct, against 14 of 17 at confidence 3. There were five errors:

- **Two false positives.** In one, a real current/power break sat just outside the alert window; in the other, the judge mistook a submeter's zero standby reading for a dropout.
- **Three misses:** two subtle drifts and one low spike.

Used as a filter on USAD's alerts, the judge keeps 21 of the 24 true segments and discards 81 of 83 false alarms, raising alert precision from 0.25 to 0.92 (Figure 8b).

This complements the ground-truth labels rather than replacing them. The LLM is good at the physical-consistency reasoning that USAD lacks: zero voltage while power is drawn, sub-meters exceeding the total, current not tracking power. But it only sees what USAD flags, so it cannot recover the 56 segments USAD missed.

Caveats. The verdicts come from one model and one pass, and the same assistant also helped build the pipeline. Prompts were label-free, but full blindness cannot be guaranteed. The repository includes a script that reruns the identical prompts through an API in a fresh context, so the result can be checked with other models or repeated runs. LLM judges have known biases (Zheng et al., 2023), and a human rater on a subset would be the natural next check.

6 Ethical Considerations

The original paper. SMD comes from a commercial data centre and is released with anonymised machine IDs and normalised metrics, so it carries little privacy risk. SWaT and WADI are gated because they describe attacks on critical infrastructure; reporting on detection is low-risk, but the attack descriptions are dual-use. The bigger concern is research integrity. Headline results that depend on point adjustment and best-F1 thresholds chosen with test labels overstate real-world performance (Kim et al., 2022; Wu and Keogh, 2023), and a deployed detector tuned to look good this way could give operators false confidence or flood them with alarms. My replication shows the effect is large: F1* drops from 0.763 to 0.528 when PA is removed.

Publishing on my new data. Under the Australian Code for the Responsible Conduct of Research (National Health and Medical Research Council et al., 2018), several principles apply:

- **Honesty and rigour (P1, P2).** I must report the unflattering numbers as prominently as the flattering ones, so this report gives every result with and without PA, alongside AUC, and states that replication was only partial.

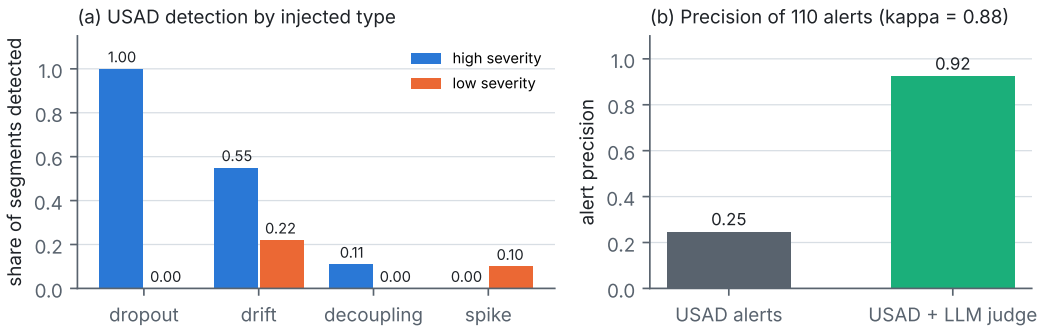


Figure 8: (a) Share of injected UCI segments USAD detects, by type and severity. (b) Precision of USAD's 110 alerts on UCI before and after filtering by the LLM judge.

- **Transparency (P3).** Code, seeds, settings, the injection log and every LLM prompt and verdict are public. The use of generative AI, both as the experimental judge and as a writing and coding aid, is declared here and in the repository.

- **Respect and data governance (P5).** The UCI data records one real household's electricity use at one-minute resolution, which can reveal when residents are home, asleep or cooking. Although it is anonymous, public and CC BY 4.0, a publication should not attempt re-identification or describe residents' routines beyond what the analysis needs. I do not redistribute the raw file; a script downloads it from the source.

- **Recognition and accountability (P6, P7).** I credit and respect the licences of the dataset creators and the USAD authors (MIT for SMD, GPL-3.0 for SKAB, CC BY 4.0 for UCI, BSD-3 for the code).

Injected anomalies are synthetic and must be labelled as such, so no one mistakes them for real faults. The LLM judge raises an accountability issue of its own. Its 0.92 precision is attractive, but if it were used to silently suppress alarms in a real system, a missed fault (it missed 3 of 27 true alerts here) would have no human owner. A responsible design keeps a human in the loop and logs the judge's rationale.

7 Conclusions and Reflections

Replication was partially successful. The official USAD code runs unmodified on all 28 SMD machines and trains stably and quickly, but it reaches F1* = 0.763 against the paper's 0.946. Without per-dataset tuning and a fully specified protocol, I could not close the gap, and removing point adjustment drops F1* to 0.528.

On new data, USAD generalises unevenly. On SKAB, it ranks anomalies well when test conditions match training but is near chance under regime shift. On the constructed household dataset, it catches gross failures such as all-channel dropouts and large drifts but misses spikes and correlation breaks, the cases a multivariate method should handle best. In both datasets, point adjustment made performance look better than ranking quality supports.

An LLM judge reviewed alerts well ($\kappa = 0.88$) and could make USAD far more usable as a filter, with alert precision rising from 0.25 to 0.92.

Reflections.

- **Report both settings from the start.** Seeing how far numbers move with point adjustment, I would now report AUC and non-PA F1 from day one rather than treating them as secondary.

- **Get more from the constructed data.** Constructing data by injection gave clean, fine-grained labels and was far more informative than I expected. I would add a second household and compare severities on a continuous scale.

- **Tune, and run more seeds.** A small hyperparameter search and several seeds per machine would have been the most valuable use of extra time, because they address the largest likely causes of the replication gap.

- **Annotation is subtle even with an LLM.** One of the judge's errors was a pattern just outside the alert window. Deciding what counts

as "the alert" turned out to matter as much as the judge itself.

References

Chuadhry Mujeeb Ahmed, Venkata Reddy Palleti, and Aditya P. Mathur. 2017. WADI: A water distribution testbed for research in the design of secure cyber physical systems. In *Proceedings of the 3rd International Workshop on Cyber-Physical Systems for Smart Water Networks*, pages 25–28.

Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A. Zuluaga. 2020. USAD: Unsupervised anomaly detection on multivariate time series. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3395–3404.

Julien Audibert, Pietro Michiardi, Frédéric Guyard, Sébastien Marti, and Maria A. Zuluaga. 2022. Do deep neural networks contribute to multivariate time series anomaly detection? *Pattern Recognition*, 132:108945.

Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

Georges Hebrail and Alice Berard. 2012. Individual household electric power consumption. UCI Machine Learning Repository.

Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Soderstrom. 2018. Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 387–395.

Vincent Jacob, Fei Song, Arnaud Stiegler, Bijan Rad, Yanlei Diao, and Nesime Tatbul. 2021. Exathlon: A benchmark for explainable anomaly detection over time series. *Proceedings of the VLDB Endowment*, 14(11):2613–2626.

Iurii D. Katser and Vyacheslav O. Kozitsin. 2020. Skoltech anomaly benchmark (SKAB). Kaggle.

Siwon Kim, Kukjin Choi, Hyun-Soo Choi, Byunghan Lee, and Sungroh Yoon. 2022. Towards a rigorous evaluation of time-series anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 7194–7201.

Dan Li, Dacheng Chen, Baihong Jin, Lei Shi, Jonathan Goh, and See-Kiong Ng. 2019. MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks. In *International Conference on Artificial Neural Networks (ICANN)*, pages 703–716.

Aditya P. Mathur and Nils Ole Tippenhauer. 2016. SWaT: A water treatment testbed for research and training on ICS security. In *2016 International Workshop on Cyber-physical Systems for Smart Water Networks (CySWater)*, pages 31–36.

National Health and Medical Research Council, Australian Research Council, and Universities Australia. 2018. Australian code for the responsible conduct of research. Commonwealth of Australia, Canberra.

Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. 2019. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2828–2837.

Renjie Wu and Eamonn J. Keogh. 2023. Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE Transactions on Knowledge and Data Engineering*, 35(3):2421–2429.

Haowen Xu, Wenxiao Chen, Nengwen Zhao, Zeyan Li, Jiahao Bu, Zhihan Li, Ying Liu, Youjian Zhao, Dan Pei, Yang Feng, et al. 2018. Unsupervised anomaly detection via variational auto-encoder for seasonal KPIs in web applications. In *Proceedings of the 2018 World Wide Web Conference*, pages 187–196.

Jiehui Xu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. Anomaly transformer: Time series anomaly detection with association discrepancy. In *International Conference on Learning Representations*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*.

A Data Snippets

```
Raw UCI file (one-minute readings, ';'-separated):
Date;Time;Global_active_power;
  Global_reactive_power;Voltage;
  Global_intensity;Sub_metering_1;
  Sub_metering_2;Sub_metering_3
16/12/2006;17:24:00;4.216;0.418;
  234.840;18.400;0.000;1.000;17.000

Constructed test set (10-min means + label):
datetime,Global_active_power,...,label
2009-07-01 00:00:00,0.224,0.036,243.585,
  0.96,0.0,0.0,0.6,0

Injection log (ground truth):
id,type,channel,severity,start,end,
  start_idx,end_idx,length_steps
0,drift,Sub_metering_2,high,
  2009-07-05 18:10,2009-07-06 14:30,
  685,808,123
32,decoupling,Global_intensity,high,
  2009-07-11 23:30,2009-07-12 10:10,
  1581,1646,65
```

B LLM-as-Judge Prompt and Example

The system prompt (abridged) describes the seven channels, notes that current physically tracks active power ($I/P \approx 4.1$–4.4), says that normal household load is bursty but physically consistent, and defines the four anomaly types. It asks for a JSON verdict. An abridged user prompt and answer, for item A078:

```
Alert length: 23 steps (230 min),
  starting Monday 09:40.
Global_active_power: before mean 4.35 |
  alert mean 5.14 ...
   series: 5.17 4.38 4.17 4.25 4.38 ...
Global_intensity: before mean 6.63 |
  alert mean 7.11 ...
I/P ratio series: 2.12 1.69 1.41 1.42
  1.39 ... 0.57 ...

{"verdict": "anomaly",
 "type": "decoupling", "confidence": 5,
 "rationale": "Current is far too low
  for the active power for hours (I/P
  about 0.6-2.4 instead of about 4.2),
  so current has stopped tracking
  power."}
```

The key confirms this alert as an injected high-severity decoupling. All 110 prompts, verdicts and the scoring script are in the repository.